4.K230 颜色巡线

4.K230 颜色巡线
1. 环境准备
2. 代码结构概述
3. 代码讲解
3.1 导入库
3.2 定义参数
3.3 初始化函数
3.4 PID计算
3.5 处理图像
3.6 主循环
3.7 流程图

本节将详细介绍使用 K230结合PID算法实现小车的巡线控制。

> 注：在本节中，我们只讲述K230部分
>
> 完整代码请参考：【源码汇总 / 05.Color / 04.find_line.py】

1. 环境准备

- **硬件**:
  - YAHBOOM K230视觉模块
  - 电机驱动模块
  - 电池供电
- **软件**:
  - CanMV IDE环境

2. 代码结构概述

代码主要分为以下几个部分：

1. **库导入**: 导入所需的模块。
2. **常量定义**: 定义显示参数、LAB颜色空间阈值（也就是巡线赛道的颜色）、PID参数。
3. **功能函数**: 包括传感器初始化、PID计算、图像处理等。
4. **主循环**: 获取图像，处理颜色，计算电机速度，并显示信息。

3. 代码讲解

3.1 导入库

```
import time, os, sys
from media.sensor import *
from media.display import *
from media.media import *
from ybUtils.YbUart import YbUart

uart = YbUart(9600)
```

这里导入了必要的库，包括时间模块、媒体传感器、显示和串口通信库。`YbUart`用于与电机控制器进行串口通信。

3.2 定义参数

```
DISPLAY_WIDTH = 640   # LCD显示宽度
DISPLAY_HEIGHT = 480  # LCD显示高度

THRESHOLDS = [
    ((21, 33, -15, 9, -9, 6)),    # 黑线
    ((40, 86, -44, -20, -24, 25)), # 绿
]

KP = 1
KI = 0.1
KD = 0.2

BASE_SPEED = 300
SCREEN_CENTER = DISPLAY_WIDTH // 2

prev_error = 0
integral = 0
```

- `DISPLAY_WIDTH`和`DISPLAY_HEIGHT`定义了LCD的显示尺寸。
- `THRESHOLDS`定义了LAB颜色空间的阈值，用于识别不同颜色的物体。
- PID控制的参数`KP`、`KI`、`KD`用于调整控制器的反应。
- `BASE_SPEED`设定了小车的基础速度。

3.3 初始化函数

```
def init_sensor():
    sensor = Sensor(width=1280, height=960)
    sensor.reset()
    sensor.set_framesize(width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT)
    sensor.set_pixformat(Sensor.RGB565)
    return sensor

def init_display():
    Display.init(Display.ST7701, to_ide=True)
    MediaManager.init()
```

- `init_sensor()`用于初始化摄像头传感器，设置分辨率和像素格式。
- `init_display()`初始化显示模块。

3.4 PID计算

```
def calculate_pid(target, current):
    global prev_error, integral

    error = target - current
    integral += error
    derivative = error - prev_error

    if integral > 100:
```

```
        integral = 100
    elif integral < -100:
        integral = -100

    output = KP * error + KI * integral + KD * derivative

    left_speed = BASE_SPEED - output
    right_speed = BASE_SPEED + output

    prev_error = error

    return int(left_speed), int(right_speed)
```

此函数用于计算PID控制输出：

- 计算当前误差、积分和微分。
- 限制积分的最大值以防止积分饱和。
- 计算左右电机的速度。

3.5 处理图像

```
def process_blobs(img, blobs, color):
    if not blobs:
        uart.send("$0,0#")
        return

    largest_blob = max(blobs, key=lambda b: b[4])

    target_x = SCREEN_CENTER
    current_x = largest_blob[0] + largest_blob[2] // 2
    left_speed, right_speed = calculate_pid(target_x, current_x)

    uart.send(f"${left_speed},{right_speed}#")

    img.draw_rectangle(largest_blob[0:4], color=color, thickness=4)
    img.draw_cross(largest_blob[0] + largest_blob[2]//2, largest_blob[1] +
largest_blob[3]//2, color=color, thickness=2)
    img.draw_line(SCREEN_CENTER, 0, SCREEN_CENTER, DISPLAY_HEIGHT, color=(0,
255, 0), thickness=1)
    img.draw_line(current_x, largest_blob[1], current_x, largest_blob[1] +
largest_blob[3], color=(255, 0, 0), thickness=2)
```

- `process_blobs()`处理检测到的色块。
- 找到最大的色块并计算其位置，进而计算PID输出并发送给电机。
- 绘制色块和参考线。

3.6 主循环

```
def main():
    try:
        sensor = init_sensor()
        init_display()
        sensor.run()

        clock = time.clock()
        color_index = 1
```

```
        threshold = THRESHOLDS[color_index]
        detect_color = get_closest_rgb(threshold)

        while True:
            clock.tick()
            img = sensor.snapshot()
            blobs = img.find_blobs([threshold], roi=(0, DISPLAY_HEIGHT//2,
DISPLAY_WIDTH, DISPLAY_HEIGHT//2))
            if blobs:
                process_blobs(img, blobs, detect_color)
            else:
                uart.send("$0,0#")

            fps = clock.fps()
            draw_fps(img, fps)
            print(fps)

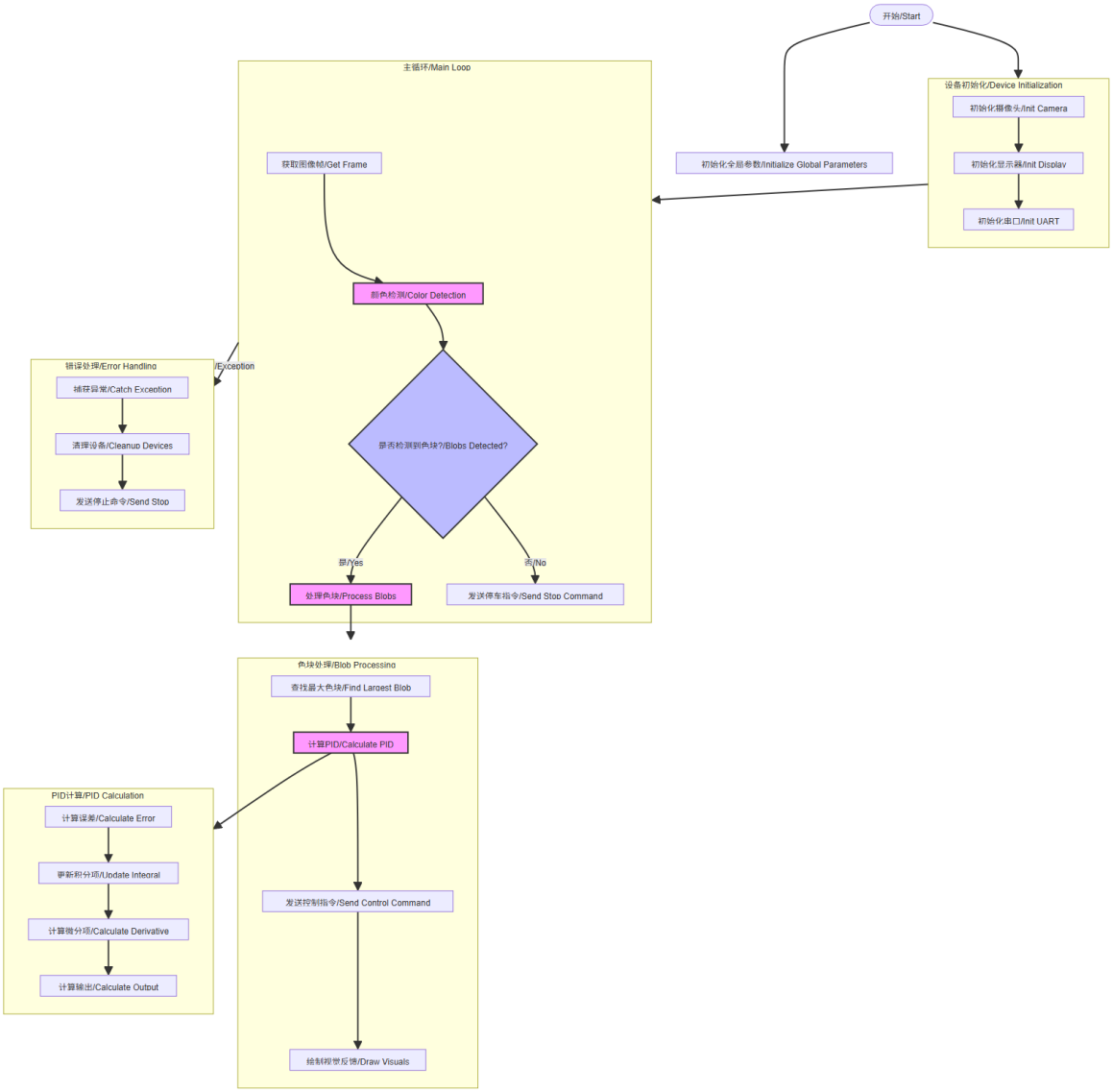
            Display.show_image(img)

    except KeyboardInterrupt as e:
        print("用户中断:", e)
    except Exception as e:
        print(f"发生错误: {e}")
    finally:
        if 'sensor' in locals() and isinstance(sensor, Sensor):
            sensor.stop()
        Display.deinit()
        MediaManager.deinit()

if __name__ == "__main__":
    main()
```

- 在`main()`函数中，初始化传感器和显示模块，并进入主循环。
- 在循环中，获取图像并检测指定颜色。根据检测结果调整电机速度，并显示图像和FPS信息。

3.7 流程图


开始/Start
设备初始化/Device Initialization
初始化摄像头/Init Camera
初始化显示器/Init Display
初始化串口/Init UART
初始化全局参数/Initialize Global Parameters
主循环/Main Loop
获取图像帧/Get Frame
颜色检测/Color Detection
是否检测到色块?/Blobs Detected?
是/Yes
否/No
处理色块/Process Blobs
发送停车指令/Send Stop Command
/Exception
错误处理/Error Handling
捕获异常/Catch Exception
清理设备/Cleanup Devices
发送停止命令/Send Stop
色块处理/Blob Processing
查找最大色块/Find Largest Blob
计算PID/Calculate PID
发送控制指令/Send Control Command
绘制视觉反馈/Draw Visuals
PID计算/PID Calculation
计算误差/Calculate Error
更新积分项/Update Integral
计算微分项/Calculate Derivative
计算输出/Calculate Output